

VESTE BEM

DOCUMENTO MESTRE CODEX V3.1

Especificação Funcional + Arquitetura Técnica + Plano de Desenvolvimento

Sistema Administrativo MVP - Veste Bem Moda Alfaiataria

Versão pronta para uso no Codex/Cursor/Claude

Sumário

1. Visão geral do produto
2. Escopo oficial do MVP
3. Tecnologias definidas
4. Papéis de usuário
5. Regras finais decididas
6. Arquitetura de pastas sugerida para o Codex
7. Identidade visual do painel
8. Módulo Dashboard
9. Módulo Vendas
10. Fluxograma completo - venda física/manual
11. Edição e cancelamento de venda
12. Módulo Pedidos
13. Fluxograma completo - pedido do site
14. Regras de status de pedido
15. Módulo Produtos
16. Módulo Estoque
17. Regras de estoque detalhadas
18. Módulo Clientes
19. Módulo Financeiro
20. Regras financeiras detalhadas
21. Nota fiscal - controle interno
22. Recibos dentro de Vendas
23. WhatsApp no MVP
24. Módulo Relatórios
25. Módulo Auditoria
26. Configurações
27. Banco de dados - visão geral
28. SQL inicial sugerido - tabelas principais
29. SQL schema
30. Permissões por módulo
31. RLS - regras conceituais

- 32. Telas e componentes - Dashboard
- 33. Telas e componentes - Vendas
- 34. Telas e componentes - Pedidos
- 35. Telas e componentes - Produtos
- 36. Telas e componentes - Estoque
- 37. Telas e componentes - Clientes
- 38. Telas e componentes - Financeiro
- 39. Telas e componentes - Relatórios
- 40. Telas e componentes - Auditoria
- 41. Integrações futuras previstas
- 42. Critérios de aceite do MVP
- 43. Prompt mestre para Codex

1. Visão geral do produto

A Veste Bem é uma loja de moda feminina especializada exclusivamente em coletes femininos. O sistema será um painel administrativo interno, acessado pelo navegador, para controlar a operação da loja desde o primeiro dia de funcionamento.

O sistema não é um e-commerce no MVP. O sistema não é um ERP completo. O sistema não possui checkout online no MVP. O sistema não emite nota fiscal no MVP. O sistema deve, porém, nascer preparado para integração futura com o site/catálogo, checkout, Pix automático, Melhor Envio, etiqueta e rastreo automático.

Objetivo do MVP:

- controlar vendas físicas, WhatsApp e Instagram;
- controlar pedidos vindos do site no futuro;
- controlar produtos, variações e estoque;
- controlar clientes;
- controlar entradas, saídas e lucro bruto estimado;
- gerar recibos sob demanda dentro da venda;
- permitir relatórios gerenciais;
- permitir auditoria apenas para administrador.

2. Escopo oficial do MVP

Menu oficial do MVP:

1. Dashboard
2. Vendas
3. Pedidos
4. Produtos
5. Estoque
6. Clientes
7. Financeiro
8. Relatórios
9. Auditoria
10. Configurações

Regra importante: não existe menu Recibos. Recibos ficam integrados dentro do módulo Vendas.

O sistema deve ser simples, rápido, seguro e funcional. Evitar transformar o MVP em ERP completo. O foco é operação real da Veste Bem.

3. Tecnologias definidas

Frontend:

- HTML
- CSS
- JavaScript puro

Backend/Banco:

- Supabase
- PostgreSQL
- Supabase Auth
- Supabase Storage apenas para imagens de produtos, não para recibos
- Row Level Security ativo

Hospedagem:

- Netlify ou Vercel

Bibliotecas recomendadas:

- supabase-js para comunicação com Supabase

- jsPDF ou pdfmake para geração de recibo em PDF sob demanda
- biblioteca leve de máscaras para telefone, CPF, CEP e moeda, se necessário

Não usar frameworks complexos no MVP, salvo se o desenvolvedor decidir conscientemente migrar para React depois. O escopo original considera HTML/CSS/JavaScript.

4. Papéis de usuário

Existem dois perfis principais:

Administrador:

- acesso total;
- pode ver custos;
- pode ver lucro;
- pode ver financeiro completo;
- pode editar venda criada;
- pode cancelar venda;
- pode editar financeiro manualmente;
- pode excluir despesas;
- pode acessar auditoria;
- pode gerenciar usuários e configurações.

Vendedor:

- pode registrar vendas;
- pode consultar estoque;
- pode cadastrar clientes;
- pode imprimir e gerar recibo;
- pode visualizar vendas próprias ou lista de vendas conforme regra definida;
- não pode ver custo;
- não pode ver lucro;
- não pode acessar auditoria;
- não pode excluir produtos;
- não pode excluir vendas;
- não pode editar financeiro.

5. Regras finais decididas

Venda pode ser editada depois de criada? Sim, somente pelo administrador.

Venda pode ser cancelada? Sim. Ao cancelar uma venda, o estoque deve ser devolvido e o financeiro correspondente deve ser estornado ou marcado como cancelado.

Pedido entregue pode voltar para outro status? Durante os primeiros 7 dias após ser marcado como entregue, pode haver ajuste administrativo. Depois de 7 dias no status Entregue, o pedido passa automaticamente ou manualmente para Finalizado. Após Finalizado, ele entra no histórico consolidado junto com vendas finalizadas da loja física, WhatsApp e Instagram.

Produto inativo aparece em vendas? Não. Produto inativo não aparece para registro de venda nem para venda futura pelo site. Deve continuar disponível apenas para histórico e relatórios.

Cliente é obrigatório em toda venda? Sim. Para casos em que o cliente não queira informar dados, criar previamente um cliente padrão chamado Cliente Diversos.

CPF é obrigatório? Não. CPF fica opcional no cadastro do cliente. O cliente pode ser salvo sem CPF. Quando houver solicitação de nota fiscal, o sistema deve indicar a necessidade de CPF, mas a emissão fiscal não faz parte do MVP.

Numeração de vendas: sequência única, começando em 00001.

Numeração de pedidos: usar a mesma sequência única das vendas. Ou seja, vendas e pedidos compartilham uma numeração operacional única. Exemplo: Venda #00001, Pedido #00002, Venda #00003. Isso facilita recibos, relatórios, auditoria e histórico.

Financeiro pode ser editado manualmente? Sim, somente pelo administrador.

Despesas podem ser excluídas? Sim, somente pelo administrador. Toda exclusão deve ser registrada na auditoria.

6. Arquitetura de pastas sugerida para o Codex

Estrutura sugerida:

```
/index.html
/login.html
/assets
/css
  styles.css
  dashboard.css
  forms.css
  tables.css
  print.css
/js
  app.js
  auth.js
  supabaseClient.js
  router.js
  utils.js
  permissions.js
  audit.js
  pdfReceipt.js
  whatsapp.js
  modules/
    dashboard.js
    sales.js
    orders.js
    products.js
    stock.js
    customers.js
    finance.js
    reports.js
    auditPage.js
    settings.js
/assets/img
  logo.png
/sql
  001_schema.sql
  002_rls.sql
  003_functions.sql
  004_seed.sql
/docs
  README.md
  SUPABASE_SETUP.md
```

O Codex deve construir o sistema em arquivos separados por responsabilidade, evitando um único arquivo gigante.

7. Identidade visual do painel

Estilo visual:

- moderno;
- minimalista;
- profissional;

- inspirado em Bling, Tiny, Omie e Nuvemshop Admin;
- não seguir o estilo editorial do site de moda.

Paleta:

- Off white para fundo geral;
- Branco para cards e tabelas;
- Azul marinho para menu lateral e elementos principais;
- Dourado apenas em detalhes, badges, ícones ou microacentos;
- Cinza claro para divisórias;
- Verde para status positivo;
- Amarelo/laranja para pendências;
- Vermelho para erros, cancelamentos e estoque zerado.

Layout:

- menu lateral fixo;
- topo com busca global, nome do usuário e botão de sair;
- área principal com cards, tabelas e filtros;
- responsivo para notebook, desktop e tablet;
- uso em celular não é prioridade, mas deve ser minimamente funcional.

8. Módulo Dashboard

Objetivo: dar visão rápida da operação e destacar pendências que exigem ação.

Cards principais:

- Vendas hoje;
- Vendas do mês;
- Faturamento do mês;
- Lucro bruto estimado, visível apenas para administrador;
- Produtos com estoque baixo;
- Notas fiscais pendentes;
- Pedidos com pendência no envio.

Regra de pedidos no dashboard:

- não mostrar todos os pedidos;
- mostrar apenas pendências operacionais relacionadas a envio;
- exemplos: pedidos pagos aguardando separação, pedidos aguardando envio, pedidos enviados sem código de rastreio, pedidos parados há muitos dias.

Seções:

- Últimas vendas;
- Últimos pedidos com pendência;
- Produtos mais vendidos;
- Despesas recentes, visível apenas para administrador;
- Alertas.

Fluxo do dashboard:

Usuário faz login -> sistema identifica perfil -> carrega indicadores permitidos -> exibe alertas -> usuário clica em um card -> sistema leva para módulo filtrado.

9. Módulo Vendas

Objetivo: registrar vendas já finalizadas de loja física, WhatsApp e Instagram, consultar histórico e gerar recibos.

Canais de venda:

- Loja Física;
- WhatsApp;

- Instagram;
- Site apenas quando pedido estiver finalizado e integrado ao histórico.

Campos de venda:

- número único;
- data e hora;
- cliente obrigatório;
- canal;
- forma de pagamento;
- desconto;
- observações;
- status: finalizada ou cancelada;
- usuário responsável;
- itens da venda.

Itens da venda:

- produto;
- cor;
- tamanho;
- quantidade;
- preço unitário;
- subtotal;
- custo unitário congelado no momento da venda para cálculo histórico de lucro.

Ações:

- registrar venda;
- ver detalhes;
- editar venda, somente admin;
- cancelar venda;
- gerar recibo;
- imprimir recibo;
- baixar PDF;
- enviar recibo por WhatsApp;
- filtrar vendas.

Regra de recibo:

- não existe módulo Recibos;
- o PDF não fica salvo no banco;
- o PDF é gerado sob demanda a partir dos dados da venda;
- se baixar, fica salvo no dispositivo do usuário;
- pode gerar novamente a qualquer momento.

10. Fluxograma completo - venda física/manual

1. Vendedor acessa Vendas.
2. Clica em Nova Venda.
3. Seleciona ou cadastra cliente. Se cliente não quiser informar dados, usar Cliente Diversos.
4. Seleciona produto ativo.
5. Seleciona cor, tamanho e quantidade.
6. Sistema valida estoque disponível.
7. Vendedor informa canal: Loja Física, WhatsApp ou Instagram.
8. Vendedor informa forma de pagamento: Pix, dinheiro ou cartão.
9. Vendedor informa desconto, se houver.
10. Sistema calcula subtotal, desconto e total.
11. Vendedor salva venda.
12. Sistema gera número único da operação.
13. Sistema baixa estoque automaticamente.
14. Sistema cria movimentação de estoque.

15. Sistema cria entrada financeira automática.
16. Sistema calcula lucro bruto estimado.
17. Sistema grava auditoria.
18. Sistema oferece ações: imprimir recibo, baixar PDF ou enviar WhatsApp.
19. Venda entra no histórico.

11. Edição e cancelamento de venda

Edição:

- permitida somente para administrador;
- deve registrar auditoria antes e depois;
- se alterar itens, o sistema deve recalcular estoque e financeiro;
- regra preferencial: para evitar erros, permitir edição apenas de observações, cliente, forma de pagamento e desconto; alteração de itens deve exigir confirmação especial.

Cancelamento:

- permitido;
- vendedor não cancela se a regra final do app restringir ao admin; recomendação: cancelamento somente admin;
- ao cancelar, devolver itens ao estoque;
- marcar entrada financeira como cancelada/estornada;
- não apagar venda do banco;
- manter venda com status Cancelada;
- registrar auditoria.

12. Módulo Pedidos

Objetivo: acompanhar pedidos vindos do site/catálogo no futuro. Site e catálogo são a mesma origem: Site.

No MVP, não deve existir botão Novo Pedido, pois o pedido virá do site quando a integração estiver pronta. A tela Pedidos deve ser uma central de acompanhamento e processamento, não de criação manual. Se for necessário criar pedido manual para teste, isso deve ficar oculto ou restrito ao administrador/desenvolvimento, não como fluxo principal.

Status do pedido:

1. Aguardando pagamento
2. Pago
3. Em separação
4. Aguardando envio
5. Enviado
6. Entregue
7. Finalizado
8. Cancelado

Regra especial:

- após 7 dias no status Entregue, o pedido passa para Finalizado;
- pedidos finalizados entram no histórico consolidado junto às vendas finalizadas.

Tela lista de pedidos:

- número;
- data;
- cliente;
- total;
- pagamento;
- status do pedido;
- envio;
- rastreio;
- ações.

Filtros:

- status;
- data;
- cliente;
- pagamento;
- envio;
- rastreio pendente.

13. Fluxograma completo - pedido do site

1. Cliente acessa o site/catálogo.
2. Escolhe colete.
3. Seleciona cor e tamanho.
4. Adiciona ao carrinho, no futuro.
5. Finaliza compra, no futuro.
6. Sistema cria pedido no banco com status Aguardando pagamento.
7. Enquanto Aguardando pagamento, não baixa estoque.
8. Pagamento é confirmado manualmente no MVP ou automaticamente no futuro.
9. Pedido muda para Pago.
10. Ao mudar para Pago, sistema baixa estoque.
11. Sistema cria movimentação de estoque.
12. Sistema cria entrada financeira automática.
13. Sistema calcula custo dos produtos vendidos e lucro bruto estimado.
14. Equipe muda pedido para Em separação.
15. Equipe separa produto.
16. Equipe muda para Aguardando envio.
17. Equipe insere transportadora, código e link de rastreio dentro do pedido.
18. Sistema muda para Enviado.
19. Equipe envia mensagem de rastreio pelo WhatsApp.
20. Pedido muda para Entregue.
21. Após 7 dias, pedido muda para Finalizado.
22. Pedido finalizado fica no histórico consolidado.

14. Regras de status de pedido

Aguardando pagamento:

- pedido criado, mas pagamento ainda não confirmado;
- não baixa estoque;
- não registra financeiro;
- pode ser cancelado.

Pago:

- pagamento confirmado;
- baixa estoque;
- registra financeiro;
- calcula lucro estimado.

Em separação:

- equipe está separando o pedido.

Aguardando envio:

- produto separado, aguardando postagem/coleta.

Enviado:

- rastreio informado;
- deve ter código e/ou link de rastreio.

Entregue:

- cliente recebeu;
- inicia contagem de 7 dias.

Finalizado:

- após 7 dias em Entregue;
- entra no histórico consolidado;
- não deve voltar status sem ação administrativa excepcional.

Cancelado:

- se cancelado antes de Pago, não altera estoque;
- se cancelado depois de Pago, devolve estoque e estorna financeiro.

15. Módulo Produtos

Objetivo: cadastrar e gerenciar coletes e suas informações comerciais.

Campos do produto:

- id;
- nome;
- descrição;
- preço de venda;
- custo;
- foto;
- status ativo/inativo;
- data de criação;
- data de atualização.

Variações:

- produto;
- cor;
- tamanho;
- quantidade;
- estoque mínimo;
- status.

Produto inativo:

- não aparece em venda;
- não aparece para pedidos futuros do site;
- continua em relatórios e histórico.

Ações:

- cadastrar produto;
- editar produto;
- inativar produto;
- excluir produto apenas se não houver histórico, preferencialmente evitar exclusão física;
- gerenciar variações.

16. Módulo Estoque

Objetivo: permitir visualização clara do estoque por produto, cor e tamanho.

Tela Estoque deve ser separada do módulo Produtos para melhor visualização.

Tabela:

- produto;
- cor;
- tamanho;

- quantidade atual;
- estoque mínimo;
- status: normal, baixo, zerado;
- última movimentação;
- ações.

Ações:

- entrada manual;
- saída manual;
- ajuste;
- ver histórico.

Regras:

- venda finalizada baixa estoque;
- pedido marcado como Pago baixa estoque;
- cancelamento de venda devolve estoque;
- cancelamento de pedido pago devolve estoque;
- ajuste manual exige motivo;
- toda movimentação gera auditoria.

17. Regras de estoque detalhadas

Entrada manual:

- usada para compra de mercadoria ou ajuste positivo;
- exige produto, variação, quantidade e motivo;
- aumenta estoque;
- registra stock_movements;
- registra audit_logs.

Saída manual:

- usada para perdas, avarias ou ajustes negativos;
- exige motivo;
- reduz estoque;
- não pode deixar estoque negativo, salvo se o admin permitir explicitamente; recomendação: bloquear estoque negativo.

Venda:

- reduz estoque no momento em que a venda é salva.

Pedido:

- reduz estoque apenas quando status muda para Pago.

Cancelamento:

- devolve as quantidades ao estoque, se já houve baixa.

Estoque baixo:

- quantidade atual menor ou igual ao estoque mínimo.

Sem estoque:

- quantidade atual igual a zero.

18. Módulo Clientes

Objetivo: manter cadastro e histórico de compras.

Campos:

- nome;
- WhatsApp;
- e-mail opcional;

- cidade;
- CPF opcional;
- observações;
- data de cadastro.

Cliente obrigatório:

- toda venda deve ter cliente;
- criar cliente padrão Cliente Diversos para vendas sem identificação.

Histórico:

- vendas;
- pedidos;
- total gasto;
- última compra.

CPF:

- não obrigatório para salvar cliente;
- necessário apenas quando houver solicitação de nota fiscal, como informação interna.

19. Módulo Financeiro

Objetivo: registrar entradas, despesas e lucro estimado.

Entradas automáticas:

- vendas finalizadas;
- pedidos marcados como Pago.

Saídas manuais:

- compra de mercadoria;
- aluguel;
- condomínio;
- internet;
- embalagem;
- etiqueta;
- tráfego pago;
- taxa de cartão;
- transporte;
- outros.

Campos de despesa:

- data;
- categoria;
- descrição;
- valor;
- forma de pagamento;
- observações;
- usuário responsável.

Administrador pode:

- criar despesa;
- editar lançamento financeiro;
- excluir despesa;
- ver lucro;
- ver custos.

Vendedor não pode:

- ver custos;
- ver lucro;

- editar financeiro;
- excluir despesa.

20. Regras financeiras detalhadas

Venda finalizada:

- cria entrada financeira do tipo Receita de Venda;
- registra faturamento bruto;
- registra desconto;
- registra custo dos produtos vendidos;
- calcula lucro bruto estimado.

Pedido Pago:

- cria entrada financeira do tipo Receita de Pedido;
- valor dos produtos e frete devem ser armazenados separadamente;
- lucro estimado considera produtos; frete deve ser tratado separadamente.

Cancelamento:

- não apagar lançamento original;
- criar lançamento de estorno ou marcar lançamento como cancelado;
- relatório deve desconsiderar cancelados.

Lucro bruto estimado:

Receita líquida dos produtos - custo dos produtos vendidos - despesas consideradas no período quando aplicável.

No MVP, o lucro é estimado, não contábil.

21. Nota fiscal - controle interno

O sistema não emite nota fiscal no MVP.

Campos:

- cliente solicitou nota fiscal;
- CPF do cliente, se disponível;
- número da nota;
- status: Pendente ou Emitida.

Regra:

- se cliente solicitou nota fiscal, destacar pendência;
- dashboard mostra notas pendentes;
- relatório de notas exibe pendentes e emitidas;
- emissão real fica fora do MVP.

22. Recibos dentro de Vendas

Não criar módulo separado de Recibos.

Dentro dos detalhes da venda:

- botão Gerar Recibo;
- botão Imprimir;
- botão Baixar PDF;
- botão Enviar WhatsApp.

Formatos:

1. 1/3 de folha A4;
2. térmica 80mm.

Conteúdo:

- VESTE BEM;
- Shopping Via Norte;
- Rua 300 - Goiânia;
- WhatsApp;
- número da venda;
- data;
- cliente;
- produto;
- cor;
- tamanho;
- quantidade;
- valor;
- forma de pagamento;
- mensagem: Obrigado pela preferência.

Armazenamento:

- sistema salva apenas os dados;
- PDF é gerado sob demanda;
- PDF não é salvo no Supabase Storage;
- se baixado, fica no dispositivo do usuário.

23. WhatsApp no MVP

No MVP, o envio pelo WhatsApp deve ser assistido, não automático via API.

Recibo:

- sistema gera PDF;
- usuário baixa ou compartilha;
- botão WhatsApp abre conversa com mensagem pronta para o número do cliente.

Rastreo:

Mensagem padrão:

Olá, [NOME]! Seu pedido Veste Bem #[NÚMERO] foi enviado.

Transportadora: [TRANSPORTADORA]

Código de rastreio: [CÓDIGO]

Você pode acompanhar por aqui:

[LINK]

Obrigada por comprar com a Veste Bem.

Automação 100% via WhatsApp Business API fica para versão futura.

24. Módulo Relatórios

Relatórios do MVP:

- produtos mais vendidos;
- produtos menos vendidos;
- canais: loja física, Instagram, WhatsApp e site;
- estoque baixo;
- sem estoque;
- financeiro por período;
- receitas;
- despesas;
- lucro estimado;

- notas fiscais pendentes;
- notas fiscais emitidas.

Não adicionar relatório específico de clientes no MVP.

25. Módulo Auditoria

Auditoria é exclusiva do administrador.

Objetivo:

- registrar ações importantes;
- permitir rastrear erros;
- proteger financeiro e estoque.

Eventos:

- login;
- logout se aplicável;
- venda criada;
- venda editada;
- venda cancelada;
- pedido alterado;
- pedido marcado como pago;
- pedido cancelado;
- produto criado;
- produto editado;
- produto inativado;
- estoque ajustado;
- despesa criada;
- despesa editada;
- despesa excluída;
- configuração alterada;
- usuário criado/editado.

Campos do log:

- id;
- data/hora;
- usuário;
- perfil;
- ação;
- módulo;
- entidade;
- id da entidade;
- dados anteriores;
- dados novos;
- IP/dispositivo se disponível.

26. Configurações

Dados da empresa:

- nome fantasia;
- endereço;
- WhatsApp;
- logo;
- mensagem padrão do recibo.

Usuários:

- listar usuários;

- definir perfil;
- ativar/inativar.

Impressão:

- formato padrão de recibo: 1/3 A4 ou 80mm;
- configurar dados do rodapé.

Backup:

- exportar CSV de produtos, estoque, vendas, clientes, financeiro e pedidos.

27. Banco de dados - visão geral

Usar PostgreSQL no Supabase.

Tabelas sugeridas:

- profiles
- operation_sequence
- customers
- products
- product_variations
- stock_movements
- sales
- sale_items
- orders
- order_items
- order_tracking
- financial_entries
- expenses
- audit_logs
- settings

Recomendação importante:

Usar uma sequência única para sales e orders por meio da tabela operation_sequence ou sequence do PostgreSQL. O campo operation_number deve ser exibido formatado com 5 dígitos: 00001, 00002, 00003.

28. SQL inicial sugerido - tabelas principais

O Codex deve adaptar este SQL ao projeto Supabase. Tipos podem ser ajustados conforme necessidade, mas a estrutura deve seguir a lógica abaixo.

29. SQL schema

```
-- Extensões
```

```
create extension if not exists "uuid-oss";
```

```
-- Perfis
```

```
create table profiles (
```

```
    id uuid primary key references auth.users(id) on delete cascade,
```

```
    name text not null,
```

```
    role text not null check (role in ('admin', 'seller')),
```

```
    active boolean default true,
```

```
    created_at timestamptz default now(),
    updated_at timestamptz default now()
);
```

-- Sequência operacional única para vendas e pedidos

```
create table operation_sequence (
    id bigint generated always as identity primary key,
    entity_type text not null check (entity_type in ('sale', 'order')),
    created_at timestamptz default now()
);
```

-- Clientes

```
create table customers (
    id uuid primary key default uuid_generate_v4(),
    name text not null,
    whatsapp text,
    email text,
    city text,
    cpf text,
    notes text,
    is_default boolean default false,
    created_by uuid references profiles(id),
    created_at timestamptz default now(),
    updated_at timestamptz default now()
);
```

-- Produtos

```
create table products (
    id uuid primary key default uuid_generate_v4(),
    name text not null,
    description text,
    sale_price numeric(10,2) not null default 0,
    cost_price numeric(10,2) not null default 0,
    image_url text,
```

```
status text not null default 'active' check (status in ('active', 'inactive')),
created_by uuid references profiles(id),
created_at timestamptz default now(),
updated_at timestamptz default now()
);
```

-- Variações

```
create table product_variations (
    id uuid primary key default uuid_generate_v4(),
    product_id uuid not null references products(id),
    color text not null,
    size text not null,
    quantity integer not null default 0,
    minimum_stock integer not null default 0,
    status text not null default 'active' check (status in ('active', 'inactive')),
    created_at timestamptz default now(),
    updated_at timestamptz default now(),
    unique(product_id, color, size)
);
```

-- Vendas

```
create table sales (
    id uuid primary key default uuid_generate_v4(),
    operation_number bigint unique not null,
    customer_id uuid not null references customers(id),
    channel text not null check (channel in ('Loja Física', 'WhatsApp', 'Instagram', 'Site')),
    payment_method text not null check (payment_method in ('Pix', 'Dinheiro', 'Cartão')),
    gross_total numeric(10,2) not null default 0,
    discount numeric(10,2) not null default 0,
    net_total numeric(10,2) not null default 0,
    total_cost numeric(10,2) not null default 0,
    estimated_gross_profit numeric(10,2) not null default 0,
    status text not null default 'finalized' check (status in ('finalized', 'cancelled')),
    invoice_requested boolean default false,
```

```

invoice_number text,
invoice_status text default 'none' check (invoice_status in ('none', 'pending', 'issued')),
notes text,
created_by uuid references profiles(id),
updated_by uuid references profiles(id),
created_at timestamptz default now(),
updated_at timestamptz default now(),
cancelled_at timestamptz
);

```

```

create table sale_items (
    id uuid primary key default uuid_generate_v4(),
    sale_id uuid not null references sales(id) on delete cascade,
    product_id uuid not null references products(id),
    variation_id uuid not null references product_variations(id),
    product_name text not null,
    color text not null,
    size text not null,
    quantity integer not null,
    unit_price numeric(10,2) not null,
    unit_cost numeric(10,2) not null,
    subtotal numeric(10,2) not null,
    total_cost numeric(10,2) not null
);

```

-- Pedidos do site

```

create table orders (
    id uuid primary key default uuid_generate_v4(),
    operation_number bigint unique not null,
    origin text not null default 'Site',
    customer_id uuid not null references customers(id),
    customer_whatsapp text,
    customer_email text,
    customer_cpf text,

```

```

products_total numeric(10,2) not null default 0,
discount numeric(10,2) not null default 0,
shipping_value numeric(10,2) not null default 0,
total numeric(10,2) not null default 0,
payment_method text,
payment_status text not null default 'pending' check (payment_status in ('pending', 'paid', 'cancelled',
'refunded')),
order_status text not null default 'Aguardando pagamento' check (order_status in ('Aguardando
pagamento', 'Pago', 'Em separação', 'Aguardando envio', 'Enviado', 'Entregue', 'Finalizado', 'Cancelado')),
postal_code text,
street text,
number text,
complement text,
neighborhood text,
city text,
state text,
shipping_method text,
carrier text,
estimated_deadline text,
internal_notes text,
paid_at timestamptz,
delivered_at timestamptz,
finalized_at timestamptz,
cancelled_at timestamptz,
created_at timestamptz default now(),
updated_at timestamptz default now()
);

```

```

create table order_items (
id uuid primary key default uuid_generate_v4(),
order_id uuid not null references orders(id) on delete cascade,
product_id uuid not null references products(id),
variation_id uuid not null references product_variations(id),
product_name text not null,
color text not null,

```

```

size text not null,
quantity integer not null,
unit_price numeric(10,2) not null,
unit_cost numeric(10,2) not null,
subtotal numeric(10,2) not null,
total_cost numeric(10,2) not null
);

```

```

create table order_tracking (
    id uuid primary key default uuid_generate_v4(),
    order_id uuid not null references orders(id) on delete cascade,
    tracking_code text,
    tracking_link text,
    carrier text,
    shipped_at timestamptz,
    estimated_delivery_date date,
    delivered_at timestamptz,
    created_at timestamptz default now(),
    updated_at timestamptz default now()
);

```

-- Estoque

```

create table stock_movements (
    id uuid primary key default uuid_generate_v4(),
    variation_id uuid not null references product_variations(id),
    movement_type text not null check (movement_type in
('entry','exit','adjustment','sale','order','cancel_sale','cancel_order')),
    quantity integer not null,
    previous_quantity integer not null,
    new_quantity integer not null,
    reason text,
    reference_type text,
    reference_id uuid,
    created_by uuid references profiles(id),
    created_at timestamptz default now()
);

```

```
);
```

```
-- Financeiro
```

```
create table financial_entries (  
    id uuid primary key default uuid_generate_v4(),  
    type text not null check (type in ('income','expense','reversal')),  
    category text not null,  
    description text,  
    amount numeric(10,2) not null,  
    status text not null default 'active' check (status in ('active','cancelled')),  
    reference_type text,  
    reference_id uuid,  
    created_by uuid references profiles(id),  
    updated_by uuid references profiles(id),  
    created_at timestamptz default now(),  
    updated_at timestamptz default now()  
);
```

```
create table expenses (  
    id uuid primary key default uuid_generate_v4(),  
    category text not null,  
    description text,  
    amount numeric(10,2) not null,  
    payment_method text,  
    expense_date date not null default current_date,  
    notes text,  
    created_by uuid references profiles(id),  
    updated_by uuid references profiles(id),  
    created_at timestamptz default now(),  
    updated_at timestamptz default now(),  
    deleted_at timestamptz  
);
```

```
-- Auditoria
```

```
create table audit_logs (  
    id uuid primary key default uuid_generate_v4(),  
    user_id uuid references profiles(id),  
    user_role text,  
    action text not null,  
    module text not null,  
    entity_type text,  
    entity_id uuid,  
    before_data jsonb,  
    after_data jsonb,  
    ip_address text,  
    user_agent text,  
    created_at timestamptz default now()  
);
```

-- Configurações

```
create table settings (  
    id uuid primary key default uuid_generate_v4(),  
    key text unique not null,  
    value jsonb,  
    updated_by uuid references profiles(id),  
    updated_at timestamptz default now()  
);
```

30. Permissões por módulo

Permissões devem ser aplicadas no frontend e no Supabase RLS. O frontend oculta botões e telas, mas a segurança real deve ficar no RLS.

Ação	Administrador	Vendedor
Dashboard - ver vendas	Sim	Sim
Dashboard - ver lucro/custos	Sim	Não
Vendas - criar	Sim	Sim
Vendas - editar	Sim	Não
Vendas - cancelar	Sim	Não recomendado
Vendas - gerar recibo	Sim	Sim
Pedidos - visualizar	Sim	Sim
Pedidos - alterar status	Sim	Sim, exceto cancelamento crítico se definido
Pedidos - cancelar	Sim	Não recomendado
Produtos - criar/editar	Sim	Não
Produtos - inativar/excluir	Sim	Não
Estoque - consultar	Sim	Sim
Estoque - entrada/ajuste manual	Sim	Não

Cientes - criar/editar	Sim	Sim
Financeiro - visualizar completo	Sim	Não
Financeiro - editar/excluir	Sim	Não
Relatórios - gerais	Sim	Limitado sem custos/lucro
Auditoria - acessar	Sim	Não
Configurações - gerenciar	Sim	Não

31. RLS - regras conceituais

Regras conceituais:

- Todo usuário autenticado e ativo pode ler dados operacionais básicos conforme perfil.
- Vendedores não podem ler cost_price, unit_cost, total_cost, estimated_gross_profit e relatórios de lucro. Se necessário, criar views públicas sem custos para vendedores.
- Apenas administradores podem inserir/editar produtos, custos, financeiro completo, configurações e auditoria.
- Apenas administradores podem ler audit_logs.
- Vendedores podem criar vendas e clientes.
- Vendedores podem consultar estoque sem custo.
- Nenhuma exclusão física crítica deve ser liberada para vendedores.

Recomendação técnica:

Criar views para vendedor:

- vw_products_seller sem custo;
- vw_sales_seller sem lucro;
- vw_stock_seller sem custo.

32. Telas e componentes - Dashboard

Componentes:

- cards de indicadores;
- tabela de últimas vendas;
- tabela de pedidos com pendência de envio;
- lista de alertas;
- produtos mais vendidos;
- despesas recentes para administrador.

Interações:

- clique em estoque baixo abre Estoque filtrado;
- clique em pedidos pendentes abre Pedidos filtrado;
- clique em notas pendentes abre Relatórios ou lista filtrada;
- clique em últimas vendas abre detalhes da venda.

33. Telas e componentes - Vendas

Componentes:

- botão Nova Venda;
- filtros: data, cliente, canal, pagamento, status;
- tabela de vendas;
- modal/formulário de venda;
- seletor de cliente;
- seletor de produto ativo;
- seletor de variação;
- resumo financeiro da venda;
- detalhes da venda;
- bloco de recibo.

Botões na venda:

- salvar;

- salvar e imprimir;
- gerar recibo;
- imprimir;
- baixar PDF;
- enviar WhatsApp;
- editar, admin;
- cancelar, admin.

34. Telas e componentes - Pedidos

Componentes:

- lista de pedidos;
- filtros por status, data, cliente, pagamento e envio;
- botão de atualizar status dentro do detalhe;
- bloco de endereço;
- bloco de itens;
- bloco de pagamento;
- bloco de rastreio;
- botão enviar rastreio WhatsApp.

Não mostrar botão Novo Pedido no fluxo normal.

Status visual:

- Aguardando pagamento: amarelo;
- Pago: azul;
- Em separação: roxo/azul;
- Aguardando envio: laranja;
- Enviado: verde;
- Entregue: verde suave;
- Finalizado: cinza;
- Cancelado: vermelho.

35. Telas e componentes - Produtos

Componentes:

- lista de produtos;
- filtros por status, cor, tamanho;
- formulário de produto;
- upload de foto;
- tabela de variações dentro do produto;
- botão adicionar variação;
- botão inativar produto.

Validações:

- nome obrigatório;
- preço obrigatório;
- custo visível apenas para admin;
- cor e tamanho obrigatórios em variação;
- não permitir duplicar mesma cor+tamanho no mesmo produto.

36. Telas e componentes - Estoque

Componentes:

- tabela de estoque por variação;
- filtro por produto, cor, tamanho e status;
- badges: normal, baixo, zerado;
- botão entrada;

- botão saída;
- botão ajuste;
- modal de motivo;
- histórico de movimentação.

Validações:

- quantidade obrigatória;
- motivo obrigatório para ajuste e saída manual;
- bloquear estoque negativo.

37. Telas e componentes - Clientes

Componentes:

- lista de clientes;
- busca por nome, WhatsApp, CPF;
- formulário de cliente;
- histórico de compras;
- botão nova venda para cliente.

Validações:

- nome obrigatório;
- CPF opcional;
- WhatsApp recomendado, mas pode ser opcional se for Cliente Diversos.

38. Telas e componentes - Financeiro

Componentes:

- cards: receitas, despesas, lucro estimado;
- filtros por período;
- tabela de lançamentos;
- botão nova despesa;
- categorias de despesa;
- edição admin;
- exclusão admin.

Vendedor não acessa dados completos de financeiro.

39. Telas e componentes - Relatórios

Relatórios:

- produtos mais vendidos;
- produtos menos vendidos;
- canais de venda;
- estoque baixo;
- sem estoque;
- financeiro por período;
- notas pendentes;
- notas emitidas.

Exportação:

- CSV quando aplicável.

40. Telas e componentes - Auditoria

Componentes:

- filtros por data, usuário, módulo, ação;

- tabela de logs;
- detalhe do log com antes/depois;
- exportar CSV se necessário.

Acesso exclusivo do administrador.

41. Integrações futuras previstas

V1 - Site integrado:

- catálogo do site lê produtos ativos e variações com estoque;
- checkout cria pedido no Supabase;
- pedido aparece no painel.

V2 - Pix automático:

- geração de cobrança Pix;
- webhook confirma pagamento;
- status muda para Pago;
- baixa estoque automática.

V2 - Melhor Envio:

- cálculo de frete;
- geração de etiqueta;
- preenchimento de transportadora e rastreio.

V3 - Rastreio automático:

- atualizar status de envio;
- notificar cliente;
- página de acompanhamento.

V3 - WhatsApp Business API:

- envio automático de recibo;
- envio automático de rastreio;
- mensagens transacionais.

42. Critérios de aceite do MVP

O MVP estará pronto quando:

- login funcionar com admin e vendedor;
- permissões funcionarem;
- produto com variações puder ser cadastrado;
- estoque puder ser consultado e ajustado;
- venda puder ser registrada;
- venda baixar estoque;
- venda registrar financeiro;
- venda gerar recibo sob demanda;
- administrador puder editar/cancelar venda;
- cliente puder ser cadastrado;
- pedido puder ser visualizado e processado quando criado no banco;
- pedido pago baixar estoque e registrar financeiro;
- dashboard mostrar indicadores e pendências de envio;
- financeiro mostrar entradas e despesas;
- relatórios básicos funcionarem;
- auditoria registrar eventos críticos;
- backup CSV básico funcionar.

43. Prompt mestre para Codex

Use este comando no Codex:

Construa o Sistema Administrativo MVP da Veste Bem seguindo integralmente este documento. Use HTML, CSS e JavaScript puro no frontend, Supabase como backend/banco/authenticação e prepare o projeto para deploy em Netlify ou Vercel. Implemente autenticação, permissões de Administrador e Vendedor, menu com Dashboard, Vendas, Pedidos, Produtos, Estoque, Clientes, Financeiro, Relatórios, Auditoria e Configurações. Não crie menu Recibos; recibos devem ficar dentro de Vendas e ser gerados sob demanda em PDF, sem salvar arquivos no Supabase. Use sequência única para vendas e pedidos começando em 00001. Implemente banco PostgreSQL, RLS, funções de estoque, financeiro, auditoria e telas conforme especificação. Priorize código limpo, separado por módulos e preparado para integração futura com site, Pix automático, Melhor Envio, rastreo e WhatsApp Business API.